

Manipal University

Name: Devanshi Rana

Email: rana.2502051097@mu.j.manipal.edu

Roll no: 2502051097

Phone: 7600927404

Branch: Manipal

Department: CSE - T1 - Batch 2

Batch: 2029

Degree: B.Tech

Scan to verify results



NeoColab_Manipal_2029_DSA Lab Course

Neocolab_Manipal_DSA_Week 10_PAH

Attempt : 2

Total Mark : 20

Marks Obtained : 20

Section 1 : Coding

1. Problem Statement

Imagine you are developing a program for an online ticket reservation system for a popular movie theatre. Customers can book tickets for movies they want to watch, and the reservation system should maintain a queue of customers waiting to book their tickets. The queue ensures that customers are served in the order they arrived on the website with each customer having a unique identifier associated with their reservation.

Write a program to perform enqueue and dequeue operations in the queue using a linked list.

Input Format

The first line of input consists of an integer N, representing the number of customers.

The second line consists of N space-separated integers, each representing the unique identifiers of the customers to be added to the queue.

Output Format

The first line of output prints "Queue: " followed by the space-separated elements of the queue.

The second line prints "Queue After Dequeue: " followed by the space-separated elements of the queue after performing the dequeue operation.

Refer to the sample output for the exact text and format.

Sample Test Case

Input: 6

14 52 63 95 68 49

Output: Queue: 14 52 63 95 68 49

Queue After Dequeue: 52 63 95 68 49

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>
typedef struct Node {
    int data;
    struct Node* next;
} Node;
Node* front = NULL;
Node* rear = NULL;
void enqueue(int val) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = val;
    newNode->next = NULL;
    if (rear == NULL) {
        front = rear = newNode;
    } else {
        rear->next = newNode;
        rear = newNode;
    }
}
```

```

}
}
void dequeue() {
    if (front == NULL) {
        return; // queue empty, nothing to remove
    }
    Node* temp = front;
    front = front->next;
    if (front == NULL) {
        rear = NULL;
    }
    free(temp);
}
void displayQueue() {
    Node* temp = front;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}
int main() {
    int N, i, val;
    scanf("%d", &N);
    for (i = 0; i < N; i++) {
        scanf("%d", &val);
        enqueue(val);
    }
    printf("Queue: ");
    displayQueue();
    printf("\n");
    dequeue();
    printf("Queue After Dequeue: ");
    displayQueue();
    printf("\n");
    return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Imagine you are developing a basic task management system for a small team of software developers. Each task is represented by an integer, where positive integers indicate valid tasks and negative integers indicate erroneous tasks that need to be removed from the queue before processing.

Write a program using the queue with a linked list that allows the team to add tasks to the queue, remove all erroneous tasks (negative integers), and then display the valid tasks that remain in the queue.

Input Format

The first line consists of an integer N, representing the number of tasks to be added to the queue.

The second line consists of N space-separated integers, representing the tasks. Tasks can be both positive (valid) and negative (erroneous).

Output Format

The output displays the following format:

For each task enqueued, print a message "Enqueued: " followed by the task value.

The last line displays the "Queue Elements after Dequeue: " followed by removing all erroneous (negative) tasks and printing the valid tasks remaining in the queue in the order they were enqueued.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

12 -54 68 -79 53

Output: Enqueued: 12

Enqueued: -54

Enqueued: 68

Enqueued: -79

Enqueued: 53

Queue Elements after Dequeue: 12 68 53

Answer

```
#include <stdio.h>
#include <stdlib.h>
```

```
typedef struct Node {
    int data;
    struct Node* next;
} Node;
```

```
typedef struct Queue {
    Node* front;
    Node* rear;
} Queue;
```

```
Queue q;
```

```
void initQueue() {
    q.front = q.rear = NULL;
}
```

```
int isEmpty() {
    return q.front == NULL;
}
```

```
void enqueue(int data) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;
    newNode->next = NULL;
    if (q.rear == NULL) {
        q.front = q.rear = newNode;
    } else {
        q.rear->next = newNode;
        q.rear = newNode;
    }
    printf("Enqueued: %d\n", data);
}
```

```
void dequeueAllNegative() {
    if (isEmpty()) {
```

```

    return;
}

// Remove negative nodes from the front
while (q.front != NULL && q.front->data < 0) {
    Node* temp = q.front;
    q.front = q.front->next;
    free(temp);
}

```

```

// If queue becomes empty
if (q.front == NULL) {
    q.rear = NULL;
    return;
}

```

```

// Traverse and remove negative nodes
Node* current = q.front;
while (current->next != NULL) {
    if (current->next->data < 0) {
        Node* temp = current->next;
        current->next = current->next->next;
        free(temp);
        if (current->next == NULL) {
            q.rear = current;
        }
    } else {
        current = current->next;
    }
}
}

```

```

void display() {
    Node* temp = q.front;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

```

```

int main() {

```

```
int n, value;
initQueue();

scanf("%d", &n);
for (int i = 0; i < n; ++i) {
    scanf("%d", &value);
    enqueue(value);
}

dequeueAllNegative();

printf("Queue Elements after Dequeue: ");
display();

return 0;
}
```

Status : Correct

Marks : 10/10